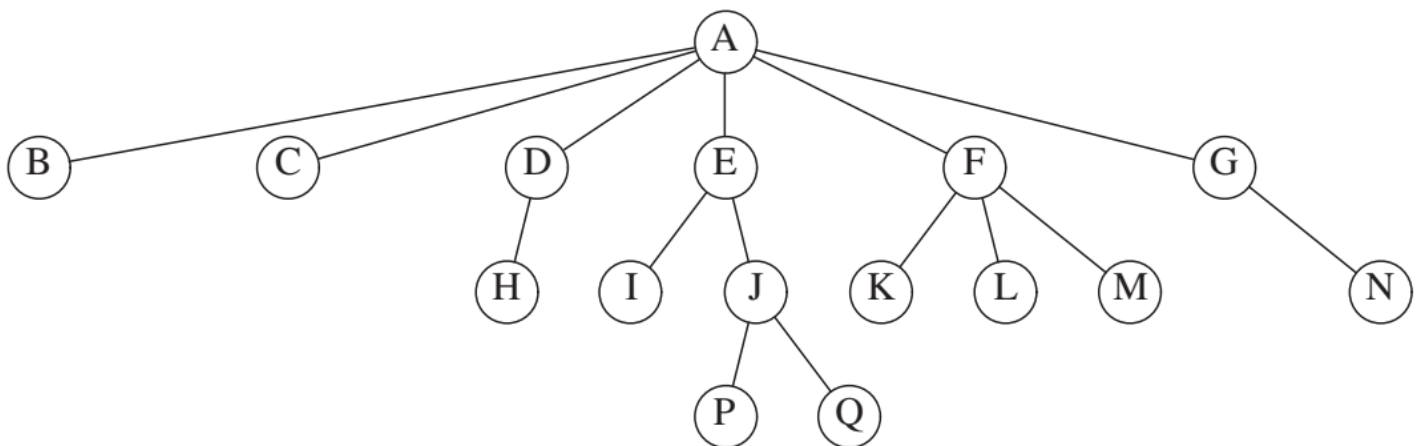
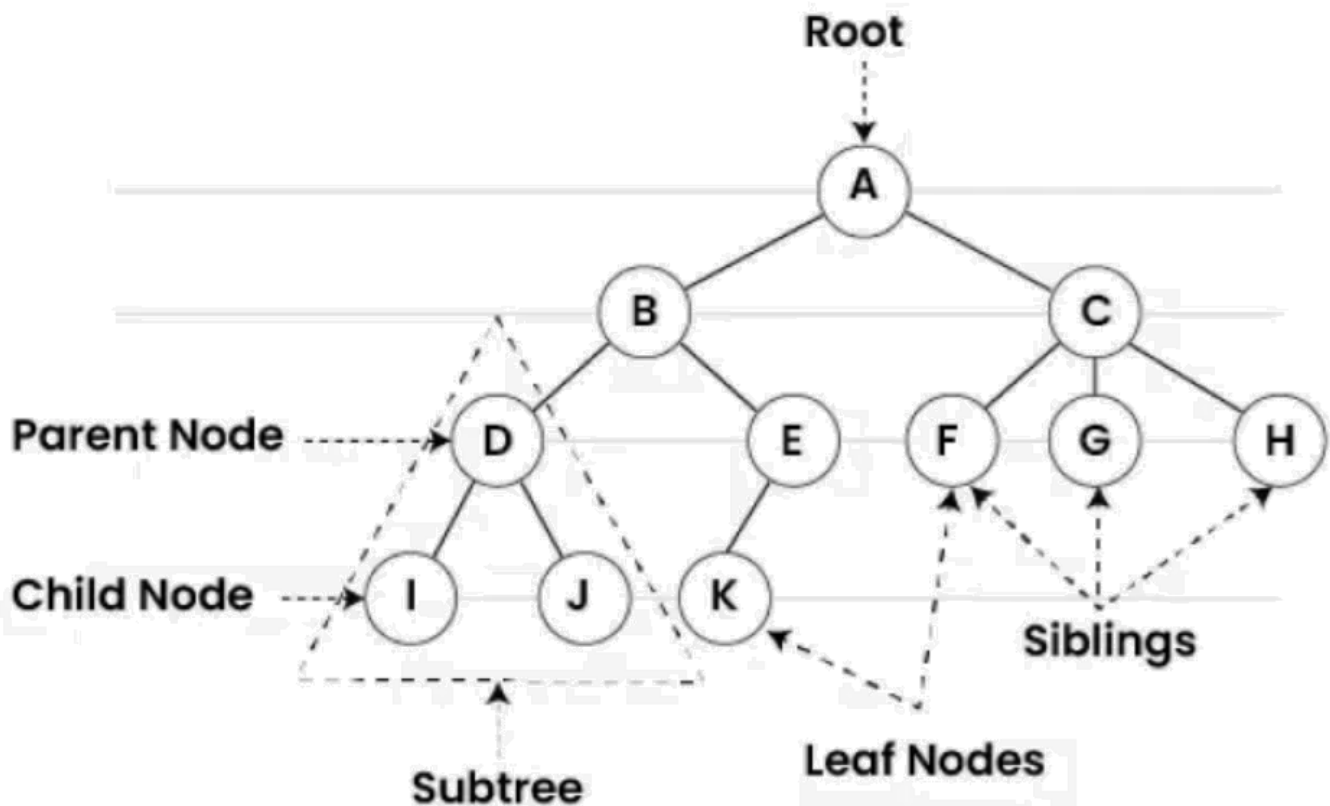




Write the path from node A to node Q A, E, J, Q

Write the path from node Q to node A Q, J, E, A

[ค่าเตือน 0 คะแนน] กฎหลักการเขียน Path ของ อ.ประดิษฐ์:

* กฎหลักของ ดร.ประดิษฐ์ พิทักษ์เสถียรกุล ในการสอบและผลงาน:

1. ห้ามใส่เครื่องหมายลูกศรเด็ดขาด! (ห้ามเขียน A -> E -> J -> Q หรือ Q <- J <- E <- A) ได้ 0 คะแนนทันที
2. ต้องเขียนคั่นด้วยเครื่องหมายจุลภาค (Comma) เท่านั้น: A, E, J, Q และ Q, J, E, A

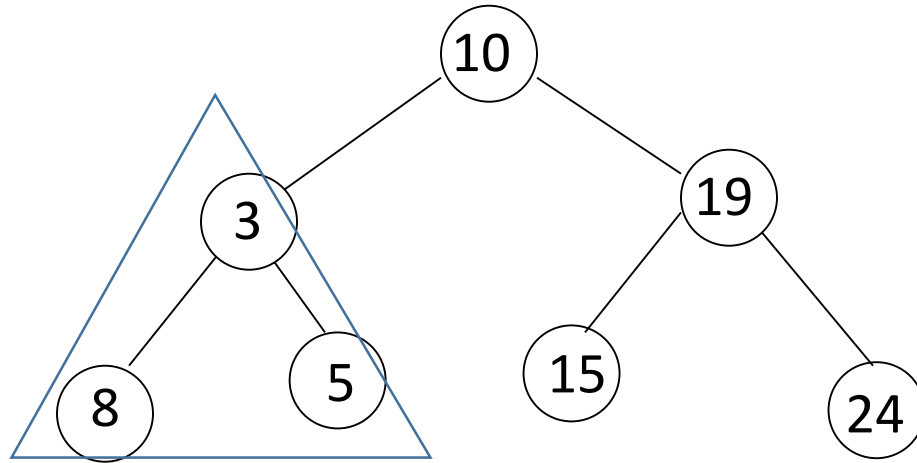
* เหตุผลทางทฤษฎี: นิยามของ Path ใน Data Structures คือ 'A sequence of nodes' (ลำดับของโหนด)

ไม่ใช่ State Transition จึงห้ามใส่ลูกศรบอกทิศทาง

Pre Order: root left right

In Order: left root right

Post Order: left right root



Pre Order : 10, 3, 8, 5, 19, 15, 24

In Order : 8, 3, 5, 10, 15, 19, 24

Post Order : 8, 5, 3, 15, 24, 19, 10

[หลักการท่องต้นไม้แบบลึก DFS Traversal ทั้ง 3 แบบ]:

* **Pre-order (Root -> Left -> Right):** ประมวลผล Root ก่อน แล้วท่อง Subtree ซ้ายและขวา

-> โหนด 10 -> (กิ่งซ้าย: 3, 8, 5) -> (กิ่งขวา: 19, 15, 24) = 10, 3, 8, 5, 19, 15, 24

* **In-order (Left -> Root -> Right):** ท่องกิ่งซ้ายจนสุด ประมวลผล Root แล้วค่อยท่องกิ่งขวา

-> (กิ่งซ้าย: 8, 3, 5) -> โหนด 10 -> (กิ่งขวา: 15, 19, 24) = 8, 3, 5, 10, 15, 19, 24

* **Post-order (Left -> Right -> Root):** ท่องกิ่งซ้ายและขวาให้เสร็จก่อน แล้วประมวลผล Root ลำดับสุดท้าย

-> (กิ่งซ้าย: 8, 5, 3) -> (กิ่งขวา: 15, 24, 19) -> โหนด 10 = 8, 5, 3, 15, 24, 19, 10

-> สำคัญ: In-order traversal ของ Binary Search Tree จะได้ผลลัพธ์เรียงลำดับจากน้อยไปมากเสมอ

[ฟังก์ชันมาตรฐานในภาษา Python และความซับซ้อน Time Complexity]:

* โค้ดภาษา Python สำหรับการท่อง Binary Tree แบบ Recursive:

```
def preorder(root): return [root.val] + preorder(root.left) + preorder(root.right) if root else []
```

```
def inorder(root): return inorder(root.left) + [root.val] + inorder(root.right) if root else []
```

```
def postorder(root): return postorder(root.left) + postorder(root.right) + [root.val] if root else []
```

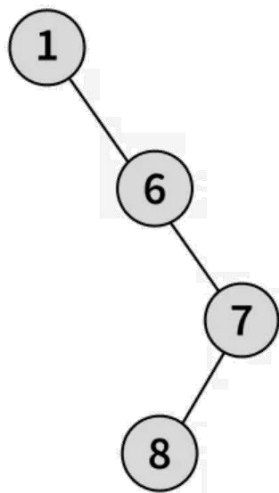
-> ความซับซ้อนเชิงเวลา: $O(N)$ ทุกวิธี เพราะต้องเยี่ยมชมครบทุก N โหนดในต้นไม้

For example 1: Construct a binary tree by using pre-order and in-order sequences given below. Write the output in post-order traversal.

In-order: 1, 6, 8, 7

Pre-order: 1, 6, 7, 8

Explanation: The tree will look like this:



Output is : ? 8, 7, 6, 1

For example 2:

Create an Expression Tree from the following infix Expression.

$(a+(b*c))+(((d*e)+f)*g)$

[วิเคราะห์ Reconstruct Tree ทีละขั้นตอน]:

- * 1. Pre-order ตัวแรกคือ 1 -> 1 เป็น Root
- * 2. In-order: 1 อยู่หน้าสุด -> ไม่มีกิ่งซ้าย, ขวา คือ [6, 8, 7]
- * 3. Pre-order ถัดไป 6 -> 6 เป็น Root กิ่งขวา
- * 4. Pre-order 7, In-order 8 มาก่อน 7 -> 8 อยู่ซ้ายของ 7
-> Post-order (Left, Right, Root): 8 -> 7 -> 6 -> 1

[กลยุทธ์โครงสร้าง Expression Tree และผลการท่อง Traversal]:

* โครงสร้าง Expression Tree: Root หลักคือ '+' เชื่อมระหว่าง 2 วงเล็บใหญ่

- กิ่งซ้าย: '+' โดยมีลูกซ้ายคือ 'a' และลูกขวาคือ '*' (b, c)
- กิ่งขวา: '*' โดยมีลูกขวาคือ 'g' และลูกซ้ายคือ '+' ('*(d, e), f)

* การท่องเพื่อตรวจสอบผลลัพธ์ (Verifications):

- Pre-order: + + a * b c * + * d e f g
- Post-order: a b c * + d e * f + g * +

-> กฎเหล็ก: โอเปอเรเตอร์ที่มี Precedence ต่ำสุดนอกวงเล็บ จะกลายเป็น Root สูงสุดของ Tree เสมอ